

Développement Web et Accès aux Bases de Données (PHP/PDO)

Matisse
kra
bts sio

Introduction et Contexte du Projet	1
Réalisations Techniques	1
1. Extraction et Affichage Structuré des Données	1
2. Factorisation et Mise en Place de la Sécurité	2
3. Interactivité et Filtrage Dynamique	3
4. Insertion Sécurisée de Données	3
Bilan et Compétences Validées	4

Introduction et Contexte du Projet

Dans le cadre de ma première année de formation en BTS SIO, j'ai réalisé ce projet pratique visant à lier une interface web à un système de gestion de bases de données. Après avoir posé les bases de la logique algorithmique avec les pseudo-codes et structuré des interfaces en HTML et CSS, ce TP m'a permis de mettre en œuvre le langage PHP pour rendre un site dynamique. C'est une excellente étape de transition pour consolider la logique procédurale avant d'aborder des langages orientés objet comme Java.

Le cas d'étude portait sur la gestion du personnel d'une entreprise via une base de données MySQL constituée de deux tables relationnelles : **service** et **employe**. L'objectif était de développer une application web permettant de lire, filtrer et ajouter des données de manière sécurisée.

Réalisations Techniques

1. Extraction et Affichage Structuré des Données

La première étape a consisté à interroger la base de données pour extraire la liste des employés. J'ai d'abord implémenté un affichage basique sous forme de liste à puces dans le fichier presentation.php.

Légende : Premier affichage brut des données de la table employé

```
presentation.php
1 <!doctype html> Add a new line at the end of this file.
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>Liste des employés</title>
6 </head>
7 <body>
8   <h1>Liste des employés</h1>
9   <ul>
10     <?php
11       try {
12         // 1. Connexion à la base de données
13         $bdd = new PDO('mysql:host=localhost;dbname=personnel;charset=utf8', 'personnel_util', 'secret'); // Revoke and cha
14
15         // 2. Exécution de la requête SQL
16         $reponse = $bdd->query('SELECT NomEmp1, PrenomEmp1 FROM employe');
17
18         // On récupère toutes les lignes d'un coup dans un tableau
19         $employes = $reponse->fetchAll(PDO::FETCH_ASSOC);
20
21         // 3. Affichage avec la boucle foreach
22         foreach ($employes as $employe) {
23           echo '<li>' . htmlspecialchars($employe['PrenomEmp1']) . ' ' . htmlspecialchars($employe['NomEmp1']) . '</li>';
24         }
25
26         // Affichage du nombre total
27         echo "<strong>Nombre d'employés : " . count($employes) . "</strong>";
28       } catch (Exception $e) {
29         // En cas d'erreur (mauvais mot de passe, etc.)
30         die('Erreur : ' . $e->getMessage());
31       }
32     }
33   </ul>
34 </body>
35 </html>
```

Matisse kra
bts sio

Pour répondre aux standards d'ergonomie web, j'ai ensuite fait évoluer cette structure vers un tableau HTML complet généré dynamiquement en PHP, offrant une bien meilleure lisibilité des informations.

Légende Mise en forme des données via une structure en tableau HTML

```
<body>
  <h1>Liste des employés</h1>
  <table>
    <thead>
      <tr>
        <th>Matricule</th>
        <th>Prénom</th>
        <th>Nom</th>
      </tr>
    </thead>
    <tbody>
      <?php
      try {
        // 1. Connexion à la base de données
        $bdd = new PDO('mysql:host=localhost;dbname=personnel;charset=utf8', 'personnel_util', 'secret');

        // 2. Exécution de la requête SQL
        $reponse = $bdd->query('SELECT Matricule, NomEmp1, PrenomEmp1 FROM employe');

        // On récupère toutes les lignes d'un coup dans un tableau
        $employes = $reponse->fetchAll(PDO::FETCH_ASSOC);

        // 3. Affichage avec la boucle foreach
        foreach ($employes as $employe) {
          echo '<tr>';
          echo '<td>' . htmlspecialchars($employe['Matricule']) . '</td>';
          echo '<td>' . htmlspecialchars($employe['NomEmp1']) . '</td>';
          echo '<td>' . htmlspecialchars($employe['PrenomEmp1']) . '</td>';
          echo '</tr>';
        }

        // Affichage du nombre total
        echo "<strong>Nombre d'employés : " . count($employes) . "</strong>";
      } catch (Exception $e) {
        // En cas d'erreur (mauvais mot de passe, etc.)
        die('Erreur : ' . $e->getMessage());
      }
      ?>
    </tbody>
  </table>
</body>
</html>
```

2. Factorisation et Mise en Place de la Sécurité

Pour rendre le code plus maintenable et professionnel, j'ai centralisé les fonctions clés dans un fichier dédié fonctions.php :

- Centralisation de la connexion : Création de la fonction getBdd() qui instancie un objet PDO pour établir la connexion avec la base de données.
- Protection contre les failles XSS : Développement d'une fonction escape() utilisant htmlspecialchars() pour nettoyer systématiquement toutes les données extraites avant leur affichage HTML.

Matisse kra
bts sio

Légende : Fichier de configuration contenant les fonctions de sécurité et de connexion

```
1 <?php Add a new line at the end of this file.
2 // fonctions.php
3
4 /**
5  * Initialise la connexion à la base de données
6  */
7 function getBdd()
8 {
9     return new PDO('mysql:host=localhost;dbname=personnel;charset=utf8', 'personnel_util', 'secret', array(PDO::ATTR_ERRMODE =>
10 })
11
12 /**
13  * Sécurise les données contre les injections XSS
14  */
15 function escape($valeur)
16 {
17     return htmlspecialchars($valeur, ENT_QUOTES, 'UTF-8', false);
18 }
19 ?> Remove this closing tag ">".
```

J'ai ensuite refactorisé mes scripts précédents pour intégrer ces bonnes pratiques.

Légende Appel des fonctions centralisées via require_once

```
<?php require_once 'fonctions.php'; ?> Add a new line at the end of this file.
<!doctype html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Tableau des employés</title>
</head>
<body>
    <h1>Tableau des employés</h1>
    <table>
        <thead>
            <tr>
                <th>Matricule</th>
                <th>Nom</th>
                <th>Prénom</th>
            </tr>
        </thead>
        <tbody>
            <?php
            try {
                // On appelle la fonction centralisée au lieu de recréer un objet PDO
                $bdd = getBdd();
                $reponse = $bdd->query('SELECT Matricule, NomEmp1, PrenomEmp1 FROM employee');
                $employees = $reponse->fetchAll(PDO::FETCH_ASSOC);
                foreach ($employees as $employee) {
                    echo '<tr>';
                    // Utilisation de la fonction de sécurité escape()
                    echo '<td>' . escape($employee['Matricule']) . '</td>';
                    echo '<td>' . escape($employee['NomEmp1']) . '</td>';
                    echo '<td>' . escape($employee['PrenomEmp1']) . '</td>';
                    echo '</tr>';
                }
            } catch (Exception $e) {
                die('Erreur : ' . $e->getMessage());
            }
            ?>
        </tbody>
    </table>
    <p><strong>Total : <?php echo count($employees); ?> employé(s)</strong></p>
</body>
</html>
```

3. Interactivité et Filtrage Dynamique

Afin de rendre l'application interactive, j'ai mis en place un système de filtrage. J'ai créé un formulaire contenant une liste déroulante <select> dont les options (<option>) sont générées dynamiquement en interrogeant la table service.

Légende : Génération dynamique des options du formulaire à partir de la BD

```
<?php require_once 'fonctions.php'; ?> Add a new line at the end of this file.
<!doctype html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Tableau des employés</title>
</head>
<body>
  <h1>Choix d'un service</h1>
  <form action="employeService.php" method="post">
    <label for="service">Service :</label>
    <select id="service" name="service">
      <?php
      try {
        // On appelle la fonction centralisée au lieu de recréer un objet PDO
        $bdd = getBdd();

        // Récupération des données de la table service
        $reponseservices = $bdd->query('SELECT CodeServ, DesiServ FROM service');
        $services        = $reponseservices->fetchAll(PDO::FETCH_ASSOC);
        foreach ($services as $service) {
          echo '<option value="' . escape($service['CodeServ']) . '">' . escape($service['DesiServ']) . '</option>';
        }
      } catch (Exception $e) {
        die('Erreur : ' . $e->getMessage());
      }
      ?>
    </select>
    <br>
    <input type="submit" value="Afficher les employés">
  </form>
</body>
</html>
```

Lors de la soumission, la page de traitement récupère le choix de l'utilisateur et exécute une requête SQL intégrant une clause JOIN (jointure) entre les tables employe et service afin de n'afficher que les salariés appartenant au service ciblé.

Légende Requête SQL avec jointure pour filtrer les résultats

```
employeeService.php
1  <?php require_once 'fonctions.php'; ?>
2  <!doctype html>
3  <html lang="fr">
4  <head>
5      <meta charset="UTF-8">
6      <title>Tableau des employés</title>
7  </head>
8  <body>
9      <h1>Choix d'un service</h1>
10     <table>
11         <thead>
12             <tr>
13                 <th>Matricule</th>
14                 <th>Nom</th>
15                 <th>Prénom</th>
16             </tr>
17         </thead>
18         <tbody>
19     <?php
20     try {
21         $bdd = getBdd();
22         $reponse = $bdd->query('SELECT Matricule, NomEmp1, PrenomEmp1 FROM employee join service on employee.ServEmp1 = service.Code');
23         $employees = $reponse->fetchAll(PDO::FETCH_ASSOC);
24         foreach ($employees as $employee) {
25             echo '<tr>';
26             // Utilisation de la fonction de sécurité escape()
27             echo '<td>' . escape($employee['Matricule']) . '</td>';
28             echo '<td>' . escape($employee['NomEmp1']) . '</td>';
29             echo '<td>' . escape($employee['PrenomEmp1']) . '</td>';
30             echo '</tr>';
31         }
32     } catch (Exception $e) {
33         die('Erreur : ' . $e->getMessage());
34     }
35     ?>
36     </tbody>
37 </table>
38 <p><strong>Total : <?php echo count($employees); ?> employé(s)</strong></p>
39 </body>
40 </html>
41
```

4. Insertion Sécurisée de Données

La dernière grande phase du TP a été la conception de formulaires permettant d'alimenter la base de données :

- Ajout d'un service : Développement d'un formulaire de saisie classique pour ajouter une nouvelle entité dans la table service.

Légende : Code du formulaire et de la requête d'insertion du service

```
<?php
try {
    $bdd = getBdd();
    $reponse = $bdd->query('SELECT CodeServ, DesiServ FROM service');
    $services = $reponse->fetchAll(PDO::FETCH_ASSOC);
    foreach ($services as $service) {
        echo '<tr>';
        // Utilisation de la fonction de sécurité escape()
        echo '<td>' . escape($service['CodeServ']) . '</td>';
        echo '<td>' . escape($service['DesiServ']) . '</td>';
        echo '</tr>';
    }
} catch (Exception $e) {
    die('Erreur : ' . $e->getMessage());
}
?>

</tbody>
</table>
<h1>liste des service</h1>
<form action="ajoutservice.php" method="post">
    <label for="CodeServ">Code du service :</label>
    <input type="text" id="CodeServ" name="CodeServ"><br>
    <label for="DesiServ">Désignation du service :</label>
    <input type="text" id="DesiServ" name="DesiServ"><br>
    <input type="submit" value="Ajouter un service">
</form>
<?php
if (isset($_POST['CodeServ']) && isset($_POST['DesiServ'])) {
    try {
        $bdd = getBdd();
        $stmt = $bdd->prepare('INSERT INTO service (CodeServ, DesiServ) VALUES (:CodeServ, :DesiServ)');
        $stmt->execute([
            ':CodeServ' => $_POST['CodeServ'],
            ':DesiServ' => $_POST['DesiServ']
        ]);
        echo '<p>Service ajouté avec succès !</p>';
    } catch (Exception $e) {
        die('Erreur : ' . $e->getMessage());
    }
}
```

Le traitement valide la présence des données transmises par la méthode HTTP POST et confirme l'insertion via un message à l'utilisateur.

Légende : Résultat visuel confirmant la réussite de l'insertion

liste des service

CodeServ Désignation

s01	Fabrication
s02	Emballage
s03	Commercial
s04	Administration

liste des service

Code du service :

Désignation du service :

[Ajouter un service](#)

Service ajouté avec succès !

Ajout d'un employé (Données complexes) : Création d'un formulaire avancé intégrant des champs textes, une case à cocher pour gérer un statut booléen (cadre oui/non) et une liste déroulante dynamique pour l'affectation du service.

Légende : Formulaire complexe générant une liste d'affectation dynamique

```
<label for="NomEmp1">Nom :</label>
<input type="text" id="NomEmp1" name="NomEmp1" required><br>

<label for="PrenomEmp1">Prénom :</label>
<input type="text" id="PrenomEmp1" name="PrenomEmp1" required><br>

<label for="Cadre">Cadre ?</label>
<input type="checkbox" id="Cadre" name="CodeCadre"><br>

<label for="service">Service :</label>
<select id="service" name="ServEmp1" required>
  <?php
    try {
      $bdd = getBdd();
      // La liste des services provient de la base de données
      $reponse = $bdd->query('SELECT CodeServ, DesiServ FROM service');
      while ($service = $reponse->fetch()) {
        echo '<option value="' . escape($service['CodeServ']) . '>' . escape($service['DesiServ']) . '</option>';
      }
    } catch (Exception $e) {
      echo '<option>Erreur de chargement</option>';
    }
  ?>
</select><br>

<input type="submit" value="Ajouter l'employé">
</form>

<?php if ($message) echo "<p><strong>$message</strong></p>"; ?> Add curly braces around the nested statement(s).
</body>
</html>
```


Matisse kra
bts sio

Le traitement PHP en amont sécurise l'insertion en utilisant la méthode `prepare()` de PDO. L'utilisation de cette requête préparée avec des marqueurs nominatifs permet de se prémunir efficacement contre les injections SQL.

Légende : Traitement de l'insertion via une requête préparée sécurisée

```
<?php Add a new line at the end of this file.
require_once 'fonctions.php';

$message = '';

// Traitement de l'insertion si le formulaire est soumis
if (isset($_POST['Matricule'])) {
    try {
        $bdd = getBdd();

        // Requête préparée avec les noms de colonnes du script SQL
        $stmt = $bdd->prepare('INSERT INTO employe (Matricule, NomEmp1, PrenomEmp1, CodeCadre, ServEmp1)
        VALUES (:mat, :nom, :pre, :cadre, :serv)');

        // Gestion du cadre : 'o' si coché, 'n' sinon
        $cadre = isset($_POST['CodeCadre']) ? 'o' : 'n';

        $stmt->execute([
            ':mat' => $_POST['Matricule'],
            ':nom' => $_POST['NomEmp1'],
            ':pre' => $_POST['PrenomEmp1'],
            ':cadre' => $cadre,
            ':serv' => $_POST['ServEmp1']
        ]);

        $message = "L'ajout a réussi !"; // Message de succès conforme à l'aperçu
    } catch (Exception $e) {
        $message = 'Erreur : ' . $e->getMessage();
    }
}

?>
<!doctype html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Ajout d'un employé</title>
</head>
<body>
    <h1>Ajout d'un employé</h1>

    <form action="ajoutEmploye.php" method="post">
        <label for="Matricule">Matricule </label>
        <input type="text" id="Matricule" name="Matricule" required><br>
```

Ajout d'un employé

Matricule :

Nom :

Prénom :

Cadre ? ☐

Service :

L'ajout a réussi !

Bilan et Compétences Validées

Ce projet m'a permis de valider des compétences essentielles du développement back-end :

- Architecture Client-Serveur : Compréhension du cycle de vie d'une page dynamique (HTML/CSS côté client, PHP/MySQL côté serveur).
- Manipulation de données (CRUD) : Utilisation de l'extension PDO pour se connecter et interagir avec une base de données relationnelle.
- Langage SQL : Maîtrise des requêtes de sélection simples, des jointures et des requêtes d'insertion.
- Sensibilisation à la sécurité : Application systématique des protections contre les failles d'injection (SQL et XSS).